# Movie Rental Database System

CS 157A — Final Project Report

**Team Members:**

Addison Schleigh — SID: 017595110, Email: addison.schleigh@sjsu.edu

Haitham Assaf — SID: 017558515, Email: haitham.assaf@sjsu.edu

Tyler Fabela — SID: 016347201, Email: tyler.fabela@sjsu.edu

Submission Date: 05/11/26

GitHub Link

## 1. Introduction

This project utilizes JDBC in order to create a database-based application using SQL and Java in order to show learning in databases. This is done by creating a database with a set of tables that interact with each other and themselves via CRUD operations and realistic operations within the system itself.

We chose to do a Movie Rental System, done from the perspective of a staff user, such that it has clean interactions among the customers, movies, renting a movie, returning the movie, and then being able to rate a movie. We can use CRUD operations on these tables while also being able to interact between tables with special queries

## 2. Objectives

**Primary Goals:** Our goal is to create a movie rental system designed with a relational schema with five tables in BCNF, populate it with at least fifteen rows per table, and build a Java application that can run SELECT, INSERT, UPDATE, and DELETE on every table through JDBC, among unique queries.

**Key Features:** Each table supports basic CRUD operations (Create, Read, Update, Delete), along with more unique operations per table. Staff can browse the catalog of movies, customers, rentals, returns, and ratings, search for movies and customers, register new customers, rent movies, process returns with auto-calculated late fees (though if they want to adjust the return date, it is not auto-calculated), and record ratings.

## 3. System Architecture

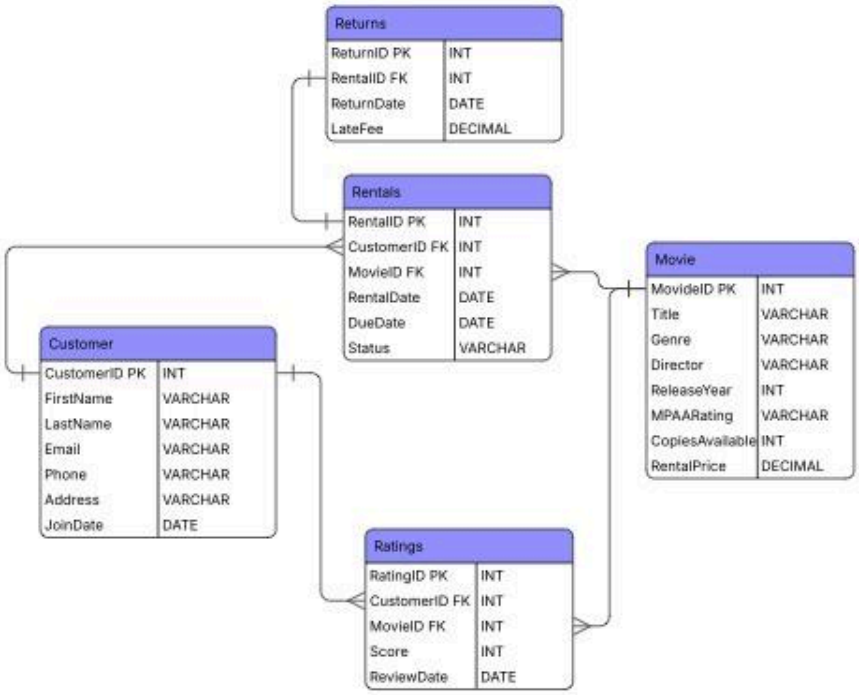
We built the system using a three-tier architecture: a Java Swing desktop GUI for the user interface (presentation layer), Java DAO classes that send SQL through JDBC (application layer), and a MySQL database normalized to BCNF (data layer).

Building it this way means each layer can change without breaking the others. For example, we could replace the Swing GUI with a web front-end, and the database code would still work the same way. Using Java DAO also helps organize the code so as not to have an overabundance of code in one area

Everything should be able to run after installation as long as you put in your mysql username and password (in DatabaseConnection) to get it running. The easiest way to do this is to simply make the password “” and run it. The driver is able to connect the SQL files to Java, and using JDBC language, such as prepared statements and executing queries, we can run SQL statements within Java and implement them in the UI

# 4. Database Design

## 4.1 Entity-Relationship Model




The system has two strong entities (Customer and Movie), two associative entities (Rentals and Ratings), and one weak entity (Returns).

Customer and Movie are the main entities. A customer can rent many movies, and a movie can be rented by many customers, so we made Rentals an associative entity that has its own attributes (rental date, due date, status). Ratings work the same way, many-to-many between Customer and Movie, essentially acting as a middle ground between the two. However, this does make it so a Customer can make many reviews for one movie, so that can be seen as an issue

Returns is a weak entity because each return only exists in relation to a rental. Without a rental, a return would not have meaning, which makes their relationship 1:1.

**Cardinalities:**

| Relationship | Connects | Cardinality |
|---|---|---|
| Makes | Customer –> Rentals | 1:M |
| For | Rentals → Movie | N:1 |
| Has | Rentals → Returns | 1:1 |
| Gives | Customer → Ratings | 1:M |
| Of | Ratings → Movie | N:1 |

### 4.2 Relational Schema

Each entity becomes a table. Relationship attributes plus the keys of the connected entities become foreign keys in the resulting tables.

| Table | Attributes |
|---|---|
| Movies | MovieID (PK), Title, Genre, Director, ReleaseYear, MPAARating, CopiesAvailable, RentalPrice |
| Customers | CustomerID (PK), FirstName, LastName, Email (UNIQUE), Phone, Address, JoinDate |
| Rentals | RentalID (PK), CustomerID (FK), MovieID (FK), RentalDate, DueDate, Status |
| Returns | ReturnID (PK), RentalID (FK, UNIQUE), ReturnDate, LateFee |
| Ratings | RatingID (PK), CustomerID (FK), MovieID (FK), Score, ReviewDate |

### 4.3 BCNF Justification

A table is in BCNF if every functional dependency X → Y has X as a superkey. We checked each table:

- Movies: only dependency is MovieID → (rest). MovieID is the primary key. BCNF holds.
- Customers: CustomerID and Email are both unique, so both are superkeys. BCNF holds.
- Rentals: only dependency is RentalID → (rest). BCNF holds.
- Returns: ReturnID and RentalID are both unique. Both are superkeys. BCNF holds.
- Ratings: only dependency is RatingID → (rest). BCNF holds.

Since every dependency is on a superkey, the schema avoids the update, insert, and delete anomalies that motivate normalization.

## 5. Functional Requirements

**Users:** Store staff are the main users. They access the system through the desktop GUI.

**Features and how they work:**

- Browse / Search Movies: Click the Movies tab to see the catalog. Type a keyword and

click Search to find matching titles. You can return to all movies by clicking Show All.
- Register a Customer: Click Add Customer on the Customers tab, fill in the form, and click OK.
- Rent a Movie: Select a movie on the Movies tab, click Rent This Movie, and pick a customer from a dropdown. The system inserts a rental and decrements CopiesAvailable.
- Process a Return: Select an active rental on the Rentals tab, click Process Return. The system auto-calculates a late fee (1.50 per day past due), inserts a Return record, marks the rental Returned, and increments CopiesAvailable. You can filter by selecting the show active button (and show all to return to viewing all)
- Submit a Rating: On the Ratings tab, click Add Rating, fill in CustomerID, MovieID, score (1-5), and date.
- Edit / Delete Records: Select a row, click Edit to open a pre-filled form. Delete asks for confirmation first. In tables with foreign keys, ON UPDATE and ON DELETE are used for referential integrity.

## 6. Non-Functional Requirements

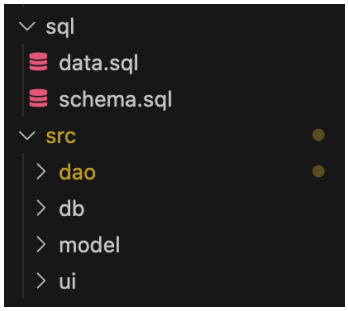
- Performance: Indexes are created automatically on primary keys and the UNIQUE Email column. Filters on Rentals and Ratings run on indexed columns.
- Security: Database credentials live in one class for easy rotation. All queries use PreparedStatement with parameter binding to prevent SQL injection.
- Maintainability: The three-tier separation means a change to the GUI does not affect the database code, and vice versa.
- Usability: Common actions are two clicks. Forms validate basic input before sending SQL. Destructive actions ask for confirmation.

## 7. Tools and Technologies

- Database: MySQL Server 9.7
- Database API: JDBC with the MySQL Connector/J driver
- Programming Language: Java (JDK 17 or later)
- UI Toolkit: Java Swing (JFrame, JTabbedPane, JTable, JOptionPane, JDialog)
- Development Environment: IntelliJ IDEA / Eclipse / VS Code
- Version Control: Git and GitHub

## 8. Implementation Details

**Code Structure**: Code is structured in two major areas: sql, which houses the schema declarations file (schema.sql) and initial data file (data.sql), and src, which houses the Java code. This Java code is organized into 4 different packages: db, to initialize the database connections via DriverManager, model packages, which give each table a Java object equivalent to use, dao, which houses the primary query functions of the database for each table, such as the CRUD operations and search, and lastly, the ui package has the ui for each table set up with Java Swing. For each package (except for db), it is organized such that each table has its own file and class to be distinct from the others.




## Important Code:

```
public class DatabaseConnection {

    private static final String URL      = "jdbc:mysql://localhost:3306/movie_rental";
    private static final String USER     = "root";
    private static final String PASSWORD = "2003";

    public static Connection getConnection() throws SQLException {
        try {
            Class.forName(className: "com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new SQLException(reason: "MySQL JDBC Driver not found.", e);
        }
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Every database call goes through JDBC. We open a Connection from DriverManager, build a PreparedStatement with parameters, run it, and close everything in a try-with-resources block so connections always get released. A typical method looks like:

```
try (Connection conn = DatabaseConnection.getConnection();
 PreparedStatement pstmt = conn.prepareStatement(sql)) {
 pstmt.setInt(1, movieId);
 try (ResultSet rs = pstmt.executeQuery()) {
 while (rs.next()) movies.add(buildMovie(rs));
 }
}
```

We use PreparedStatement with parameters (?) instead of string concatenation. This protects against SQL injection and lets the database reuse the compiled query plan.

This code establishes the JDBC connection between the Java files and the SQL files saved in MySQL, and also handles a SQLException if a driver is not found. The user will need to replace the USER and PASSWORD fields with their own sign-in information to avoid the driver not found exception.

Every DAO supports all four core SQL operations (CRUD). Here is what they look like in MovieDAO:

| Operation | Method | SQL |
|---|---|---|
| SELECT | getAllMovies() | `SELECT * FROM Movies ORDER BY MovieID` |

| | | |
|---|---|---|
| SELECT | searchByTitle(kw) | `SELECT * FROM Movies WHERE Title LIKE ?` |
| INSERT | insertMovie(m) | `INSERT INTO Movies (...) VALUES (?, ?, ...)` |
| UPDATE | updateMovie(m) | `UPDATE Movies SET ... WHERE MovieID = ?` |
| DELETE | deleteMovie(id) | `DELETE FROM Movies WHERE MovieID = ?` |

Examples of CRUD:

```java
public boolean insertCustomer(Customer c) throws SQLException {
    String sql = "INSERT INTO Customers (FirstName, LastName, Email, Phone, " +
                 "Address, JoinDate) VALUES (?, ?, ?, ?, ?, ?)";

    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(parameterIndex: 1, c.getFirstName());
        pstmt.setString(parameterIndex: 2, c.getLastName());
        pstmt.setString(parameterIndex: 3, c.getEmail());
        pstmt.setString(parameterIndex: 4, c.getPhone());
        pstmt.setString(parameterIndex: 5, c.getAddress());
        pstmt.setDate(parameterIndex: 6, c.getJoinDate());

        return pstmt.executeUpdate() > 0;
    }
}
```

Create:

As seen, this uses the INSERT INTO SQL statement with parameters of interest from a customer object created through the model class

```java
public List<Movie> getAllMovies() throws SQLException {
    String sql = "SELECT * FROM Movies ORDER BY MovieID";
    List<Movie> movies = new ArrayList<>();

    try (Connection conn = DatabaseConnection.getConnection();
         Statement  stmt = conn.createStatement();
         ResultSet  rs   = stmt.executeQuery(sql)) {

        while (rs.next()) {
            movies.add(buildMovie(rs));
        }
    }
    return movies;
}
```

Read:

As seen, reading is done by using the SELECT * statement on a table, a search will be similar, where instead they use SELECT * FROM <table> WHERE <parameter of interest> LIKE %?%;

```java
public boolean updateCustomer(Customer c) throws SQLException {
    String sql = "UPDATE Customers SET FirstName = ?, LastName = ?, Email = ?, " +
                 "Phone = ?, Address = ?, JoinDate = ? WHERE CustomerID = ?";

    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(parameterIndex: 1, c.getFirstName());
        pstmt.setString(parameterIndex: 2, c.getLastName());
        pstmt.setString(parameterIndex: 3, c.getEmail());
        pstmt.setString(parameterIndex: 4, c.getPhone());
        pstmt.setString(parameterIndex: 5, c.getAddress());
        pstmt.setDate(parameterIndex: 6, c.getJoinDate());
        pstmt.setInt(parameterIndex: 7, c.getCustomerId());

        return pstmt.executeUpdate() > 0;
    }
}
```

Update:

As seen, it uses the typical update statement UPDATE SET WHERE ID = ? using the customer object and using specified changes through the UI

```java
public boolean deleteMovie(int movieId) throws SQLException {
    String sql = "DELETE FROM Movies WHERE MovieID = ?";
    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(parameterIndex: 1, movieId);
        return pstmt.executeUpdate() > 0;
    }
}
```

Delete:

We use the typical SQL statement to delete, that being DELETE FROM <table> WHERE ID = ?, which is accessed using the delete button on a table

**Error Handling:** As seen in the aforementioned code snippets, each SQL statement is surrounded by throws SQLException statements, such that it runs into an exception if the SQL statement has an error. Also, when creating an object, we make sure that it is formatted correctly (such as DATE) by using the IllegalArgumentException with a message regarding that issue.

## 9. Testing

For our starting data, we started with 15-20 rows for each table such that it fulfills project requirements, as well as being aligned with each other, such that for each return, there is a rental, for which a rental is associated with a movie that currently exists. The tables have also been added from previous test runs, such that the data is closer to 25-30 per table

Test 1: Add new Customer(“John”,”Doe”,”john.doe@email.com”, 408-123-4567, “468 Main Street, Oakland”, “2026-05-10”), test was successful

| | | | | | | |
|---|---|---|---|---|---|---|
| 21 | John | Doe | john.doe@email.com | 408-123-4567 | 468 Main Street, Oa... | 2026-05-10 |

Test 2: Update Movie with more copies and an increased price

Before:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4 | Pulp Fiction | Crime | Quentin Tarantino | 1994 | R | 3 | 3.99 |

After:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4 | Pulp Fiction | Crime | Quentin Tarantino | 1994 | R | 5 | 5.99 |

Test 3: Rent Toy Story

Before:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 13 | Toy Story | Animation | John Lasseter | 1995 | G | 8 | 2.99 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 21 | John | Doe | john.doe@email.com | 408-123-4567 | 468 Main Street, Oa... | 2026-05-10 |

After:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 13 | Toy Story | Animation | John Lasseter | 1995 | G | 7 | 2.99 |

| | | | | | |
|---|---|---|---|---|---|
| 29 | 21 | 13 | 2026-05-10 | 2026-05-17 | Active |

Correctly shows IDs

Test 4: Delete Rental

Before:

| | | | | | |
|---|---|---|---|---|---|
| 29 | 21 | 13 | 2026-05-10 | 2026-05-17 | Active |

After:

| | | | | | |
|---|---|---|---|---|---|
| 25 | 20 | 28 | 2024-04-21 | 2024-04-28 | Active |
| 27 | 11 | 6 | 2026-05-07 | 2026-05-14 | Returned |
| 28 | 1 | 6 | 2026-05-08 | 2026-05-15 | Returned |

(Rental entry is now gone)

However, this is not the same as return, so the copy is not returned to the movies table and is lost

Test 5: Return Rental

Before:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 28 | Back to the Future | Sci-Fi | Robert Zemeckis | 1985 | PG | 5 | 3.49 |

After:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 28 | Back to the Future | Sci-Fi | Robert Zemeckis | 1985 | PG | 6 | 3.49 |

| | | | | | |
|---|---|---|---|---|---|
| 25 | 20 | 28 | 2024-04-21 | 2024-04-28 | Returned |

| | | | |
|---|---|---|---|
| 18 | 25 | 2024-05-01 | 4.50 |

One issue is that the late fee is automatically set to the current day, so for our test cases set in 2024, we have to manually input the return date and the late fee to not be out of range

So, overall, the code works as intended, although there are some minor bugs that can be updated at a later time. One issue found is that after an update from another table, you will need to select show all again to show the updated version

## 10. Challenges and Solutions

One initial challenge going into this project was with setting up the database connection, such that I had issues with the Username and Password. I solved this by using a blank password to be able to run the program. Overall, the biggest challenge was figuring out how to use MySQL and JDBC since I am not familiar with these languages and uses with Java.

## 11. Future Enhancements

- Make it from the perspective of the customer themselves, including a sign in for the system before accessing tables
- Use movie titles/customer names instead of ids for better identification
- Give detailed reviews for a movie
- Add payment ability for both renting and late fees where applicable

## 12. Conclusion

This project puts together every major topic covered in CS 157A. We started with an

entity-relationship diagram, mapped it to a relational schema, verified that every table is in BCNF, and then built a Java application that talks to the database through JDBC inside a three-tier architecture.

The result is a working movie rental application that supports all four basic SQL operations on five normalized tables. Future extensions could include: replacing the Swing GUI with a web front-end (changing only the presentation layer), adding stored procedures for late-fee calculation, or adding role-based access for staff vs. managers. Each of these could be done without changing the schema, which is the point of building it this way.

## 13. Appendix

```sql
CREATE TABLE Movies (
    MovieID         INT AUTO_INCREMENT PRIMARY KEY,
    Title           VARCHAR(150)   NOT NULL,
    Genre           VARCHAR(50)    NOT NULL,
    Director        VARCHAR(100)   NOT NULL,
    ReleaseYear     INT            NOT NULL,
    MPAARating      VARCHAR(10)    NOT NULL,
    CopiesAvailable INT            NOT NULL,
    RentalPrice     DECIMAL(5,2)   NOT NULL
);

CREATE TABLE Customers (
    CustomerID  INT AUTO_INCREMENT PRIMARY KEY,
    FirstName   VARCHAR(50)   NOT NULL,
    LastName    VARCHAR(50)   NOT NULL,
    Email       VARCHAR(100)  NOT NULL UNIQUE,
    Phone       VARCHAR(20)   NOT NULL,
    Address     VARCHAR(200)  NOT NULL,
    JoinDate    DATE          NOT NULL
);

CREATE TABLE Rentals (
    RentalID    INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID  INT          NOT NULL,
    MovieID     INT          NOT NULL,
    RentalDate  DATE         NOT NULL,
    DueDate     DATE         NOT NULL,
    Status      VARCHAR(20)  NOT NULL,
    FOREIGN KEY (CustomerID)
        REFERENCES Customers(Customer+ID)
        ON UPDATE CASCADE
        ON DELETE SET NULL,
    FOREIGN KEY (MovieID)
        REFERENCES Movies(MovieID)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);
```

```sql
CREATE TABLE Returns (
    ReturnID    INT AUTO_INCREMENT PRIMARY KEY,
    RentalID    INT           NOT NULL UNIQUE,
    ReturnDate  DATE          NOT NULL,
    LateFee     DECIMAL(5,2)  NOT NULL,
    FOREIGN KEY (RentalID)
        REFERENCES Rentals(RentalID)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);

CREATE TABLE Ratings (
    RatingID    INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID  INT   NOT NULL,
    MovieID     INT   NOT NULL,
    Score       INT   NOT NULL,
    ReviewDate  DATE  NOT NULL,
    FOREIGN KEY (CustomerID)
        REFERENCES Customers(CustomerID)
        ON UPDATE CASCADE
        ON DELETE SET NULL,
    FOREIGN KEY (MovieID)
        REFERENCES Movies(MovieID)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);
```

Schema declarations with constraints

```java
public class DatabaseConnection {

    private static final String URL      = "jdbc:mysql://localhost:3306/movie_rental";
    private static final String USER     = "root";
    private static final String PASSWORD = "2003";

    public static Connection getConnection() throws SQLException {
        try {
            Class.forName(className: "com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new SQLException(reason: "MySQL JDBC Driver not found.", e);
        }
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Driver Connection Code

```java
public boolean insertCustomer(Customer c) throws SQLException {
    String sql = "INSERT INTO Customers (FirstName, LastName, Email, Phone, " +
                 "Address, JoinDate) VALUES (?, ?, ?, ?, ?, ?)";

    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(parameterIndex: 1, c.getFirstName());
        pstmt.setString(parameterIndex: 2, c.getLastName());
        pstmt.setString(parameterIndex: 3, c.getEmail());
        pstmt.setString(parameterIndex: 4, c.getPhone());
        pstmt.setString(parameterIndex: 5, c.getAddress());
        pstmt.setDate(parameterIndex: 6, c.getJoinDate());

        return pstmt.executeUpdate() > 0;
    }
}
```

Example of Create Operations with Customer (INSERT INTO Customers …)

```java
public List<Movie> getAllMovies() throws SQLException {
    String sql = "SELECT * FROM Movies ORDER BY MovieID";
    List<Movie> movies = new ArrayList<>();

    try (Connection conn = DatabaseConnection.getConnection();
         Statement  stmt = conn.createStatement();
         ResultSet  rs   = stmt.executeQuery(sql)) {

        while (rs.next()) {
            movies.add(buildMovie(rs));
        }
    }
    return movies;
}
```

Example of Read Operation with Movies (Select * FROM Movies)

```java
public boolean updateCustomer(Customer c) throws SQLException {
    String sql = "UPDATE Customers SET FirstName = ?, LastName = ?, Email = ?, " +
                 "Phone = ?, Address = ?, JoinDate = ? WHERE CustomerID = ?";

    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(parameterIndex: 1, c.getFirstName());
        pstmt.setString(parameterIndex: 2, c.getLastName());
        pstmt.setString(parameterIndex: 3, c.getEmail());
        pstmt.setString(parameterIndex: 4, c.getPhone());
        pstmt.setString(parameterIndex: 5, c.getAddress());
        pstmt.setDate(parameterIndex: 6, c.getJoinDate());
        pstmt.setInt(parameterIndex: 7, c.getCustomerId());

        return pstmt.executeUpdate() > 0;
    }
}
```

Example of Update Operation with Customer (UPDATE Customer SET …)

```java
public boolean deleteMovie(int movieId) throws SQLException {
    String sql = "DELETE FROM Movies WHERE MovieID = ?";
    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(parameterIndex: 1, movieId);
        return pstmt.executeUpdate() > 0;
    }
}
```

Example of Delete Operation with Movie (DELTE FROM Movies …)

```java
public List<Customer> searchByName(String keyword) throws SQLException {
    String sql = "SELECT * FROM Customers " +
                 "WHERE FirstName LIKE ? OR LastName LIKE ? ORDER BY LastName";
    List<Customer> customers = new ArrayList<>();

    try (Connection conn = DatabaseConnection.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        String pattern = "%" + keyword + "%";
        pstmt.setString(parameterIndex: 1, pattern);
        pstmt.setString(parameterIndex: 2, pattern);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) customers.add(buildCustomer(rs));
        }
    }
    return customers;
}
```

Example of searching by name in Customers (Select * FROM Customers WHERE FirstName LIKE OR LastName LIKE)